

TP5 – Domaines d’horloge

Christophe Dufaza

Table des matières

Question 1	3
Architecture RTL	3
Code couleurs	3
Interface	3
Circuit	4
Question 2	5
Architecture VHDL	5
Question 3	5
Simulation	5
Question 4	6
Architecture RTL	6
Interface	6
Circuit	7
Architecture VHDL	7
Simulation	7
Question 5	8
<i>Testbench</i>	8
Question 6	8
Simulation	8
Question 7	9
<i>Pulse Stretcher</i>	9
Question 8	9
Architecture RTL	9
Architecture VHDL	10
Simulation	10
Question 9	11
IP PLL	11
Architecture RTL	11
Architecture VHDL	11
Question 10	12
Sondes ILA	12
Question 11	13
Rapports d'utilisation	13
Rapport de <i>Timing</i> (STA)	14
Intra-domaine, violations Setup	14
Inter-domaines, violations Hold	14
Question 12	15
Implémentation	15
Placement & Routage	15

Rapports de Timing (STA)	15
Question 13	15
Bitstream	15
ILA	15
Démonstration	16
<i>Patch</i> counter_unit	17

Question 1

A l'aide du module `LED_driver` du TP4, créez une architecture RTL permettant de piloter les deux LED RGB. Les LEDs RGB devront clignoter 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. En entrée des modules `LED_driver` le signal `update` ne devra pas être à 1 pendant plus d'un coup d'horloge. Il doit s'agir d'une impulsion.

Le compteur de temporisation devra compter 100 000 000 coups d'horloge (pour une fréquence d'horloge à 100MHz, cela correspond à 1s). **Dans la suite de ce TP, la valeur du compteur de temporisation ne sera pas modifiée, même lorsque les fréquences d'horloges changeront.**

Architecture RTL

Code couleurs

Les couleurs rouge, bleue, et vert sont définies à la fois¹:

- Comme codes binaires de largeur 2-bit apparaissant dans les I/O des circuits.
- Comme une suite 1-indexée, permettant d'itérer sur les couleurs dans l'ordre produisant la séquence LED demandée (rouge, bleu, vert). Une itération dans cette suite modifie un unique bit de code couleur (*gray codes*).

Table 1: Code couleurs.

colode_code	Code couleur	Couleur
COLOR_NONE	00	Réservé.
COLOR_RED	01	Rouge
COLOR_BLUE	10	Bleu
COLOR_GREEN	11	Vert

Interface

Les ports I/O du circuit pilotant deux LEDs sont donnés Table 2.

Table 2: Ports I/O circuit Question 1.

Port	Type	Signal
clk	I	Horloge
resetn	I	Reset asynchrone, active-low
LED0_R	O	Signal pilote (<i>driver</i>) LED0 canal rouge
LED0_G	O	Signal pilote (<i>driver</i>) LED0 canal vert
LED0_B	O	Signal pilote (<i>driver</i>) LED0 canal bleu
LED1_R	O	Signal pilote (<i>driver</i>) LED1 canal rouge
LED1_G	O	Signal pilote (<i>driver</i>) LED1 canal vert
LED1_B	O	Signal pilote (<i>driver</i>) LED1 canal bleu

Le circuit est paramétré par un diviseur de fréquence² D tel que la période d'un cycle allumé-éteint est $D \times T_{clk}$. Pour un clignotement de période deux secondes (LEDs allumées une seconde, éteintes une seconde), à partir d'une horloge 100 MHz, on paramètrera le circuit avec $D = \frac{2}{10^{-8}} = 2 \cdot 10^8$.

Table 3: Paramètre circuit Question 1.

Générique	Configuration
D	Diviseur de fréquence

¹Voir aussi le package `src/design/color_codes_pkg.vhd`.

²On en déduit le modulo $K = D/2 = 10^8$ du compteur de temporisation `counter_unit`.

Les codes couleur sont générés par le circuit `color_iter` décrit Fig. 1: une pulsation en entrée `cc_next` provoque une itération dans la suite des couleurs, puis, au front d'horloge suivant, une pulsation `cc_ready` indique que le code couleur correspondant peut être lu³ sur la sortie `cc_data`.



The diagram illustrates a digital circuit labeled **circuit_q1**. It features the following components and connections:

- Inputs:** `resetn` (active-low) and `clk` (clock).
- Color Iteration Block (`color_iter`):**
 - Input `cc_data` (2 bits) is connected to the `cc_next` output of the `CC_ITER` block.
 - Input `cc_ready` (1 bit) is connected to the `cc_data` input of the `DRV_LED0` block.
 - Input `resetn` is connected to the `resetn` input of the `color_iter` block.
 - Input `clk` is connected to the `clk` input of the `color_iter` block.
- Countdown Block (`r_cycle_cnt[3:0]`):**
 - Input `CE` (1 bit) is connected to the `cc_next` output of the `CC_ITER` block.
 - Input `D` (4 bits) is connected to the `cc_data` input of the `DRV_LED0` block.
 - Input `resetn` is connected to the `resetn` input of the `r_cycle_cnt` block.
 - Input `clk` is connected to the `clk` input of the `r_cycle_cnt` block.
 - Output `Q` (4 bits) is connected to a comparator (`=9`).
- LED Driver Blocks (`DRV_LED0` and `DRV_LED1`):**
 - Both blocks have a `color_code` input (2 bits) connected to the `cc_data` output of the `color_iter` block.
 - Both blocks have an `update` input (1 bit) connected to the `cc_ready` output of the `color_iter` block.
 - Both blocks have a `resetn` input connected to the `resetn` input of the `color_iter` block.
 - Both blocks have a `clk` input connected to the `clk` input of the `color_iter` block.
 - Each block has three outputs: `led_r`, `led_g`, and `led_b` (1 bit each).
- Logic Elements:**
 - A 4-to-1 multiplexer selects between the `Q` output of the `r_cycle_cnt` block and a constant value of `0..0` based on a select signal.
 - A comparator (`=9`) checks if the selected value is equal to 9.
 - An adder (`+1`) increments the output of the multiplexer by 1.

Plutôt que de dupliquer les éléments du circuit pour piloter chacune des LEDs, les deux LEDs sont pilotées *via* les mêmes signaux `cc_data` et `cc_ready`. C'est le signal `end_cycle` du *driver* connecté à la LED `LED0` qui rythme les changements de couleur⁴.

³Le registre `cc_data` sera alors stable jusqu'au prochain changement de couleur après dix clignotements, soit pendant 10×2.10^8 périodes d'horloge. Cette remarque nous permettra de mieux interpréter une violation de contrainte *Hold Time* rapportée par les rapports de synthèse.

4

Question 2

Rédigez le code VHDL correspondant à votre architecture.

Architecture VHDL

Sources VHDL:

- Description du circuit Question 1: `src/design/circuit_q1.vhd`
- Codes couleurs (*package*): `src/design/color_codes_pkg.vhd`

L'architecture de l'entité `circuit_q1` décrit littéralement le circuit Fig. 2.

Question 3

Rédigez le testbench et simulez votre design. Vérifiez que les modules réceptionnent correctement le signal `update`.

Simulation

Testbench:

- UUT: `circuit_q1`
- VHDL: `src/sim/tb_circuit_q1.vhd`
- Configuration *waveform*: `src/sim/tb_circuit_q1.wcfg`
- Pour la simulation, les circuits `led_driver` sont configurés pour un cycle allumé-éteint de 10 périodes d'horloge (5 périodes LEDs allumées, 5 périodes LEDs éteintes): $D = 10$, $K = 5$.

L'objet de la simulation est de vérifier:

- Les LEDs clignotent simultanément, avec la même couleur.
- Les changements de couleur apparaissent après dix clignotement des LEDs.
- Le stimulus `update` est une pulsation d'un cycle d'horloge.

Le rythme et les couleurs de clignotement des LEDS sont vérifiables (observables) Fig. 3.

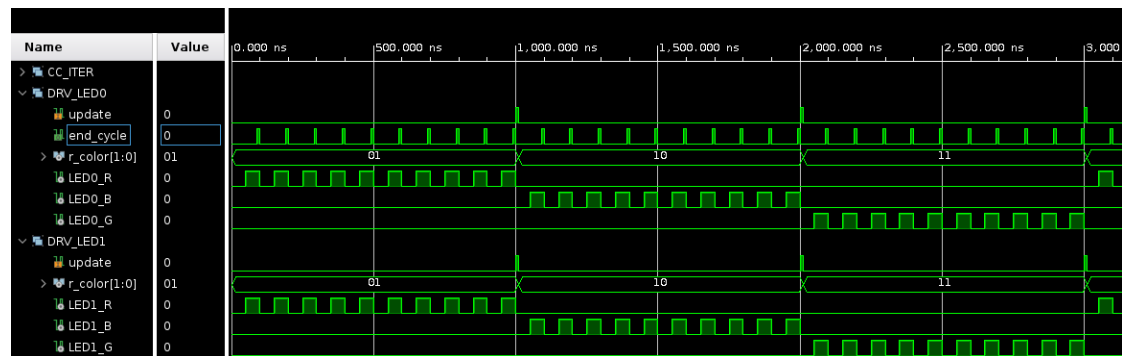


Figure 3: Chronographe `tb_circuit_q1` (3200 ns).

La durée du stimulus `update` est vérifiée (observée) Fig. 4.

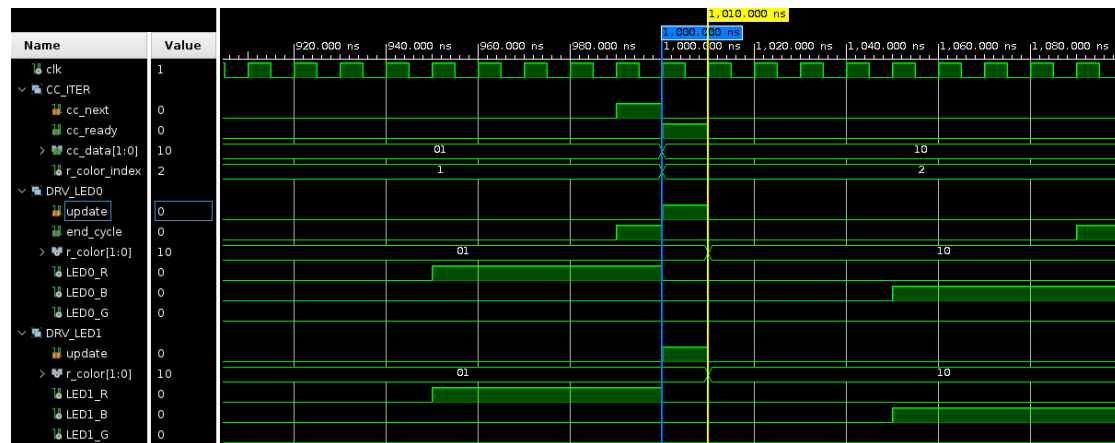


Figure 4: Stimulus `update`.

Question 4

Modifiez votre design pour gérer deux signaux d'horloge différents. La première horloge, `clkA`, est associé à la logique en dehors des modules `LED_driver` et au module `LED_driver` de la LED0. La deuxième horloge, `clkB`, est associé au module `LED_driver` de la LED1. Le changement de couleur des deux LEDs à lieu lorsque la LED0 à clignoté 10 fois.

Architecture RTL

L'architecture RTL du circuit comprend maintenant deux domaines d'horloge:

- Domaine A, signal d'horloge `clk_A`: DRV_LED0, CC_ITER, et `r_cycle_cnt`.
- Domaine B, signal d'horloge `clk_B`: DRV_LED1

Interface

Les ports I/O du circuit modifié sont donnés Table 4.

Table 4: Ports I/O circuit Question 4.

Port	Type	Signal
<code>clk_A</code>	I	Horloge domaine A
<code>clk_B</code>	I	Horloge domaine B
<code>resetn</code>	I	Reset asynchrone, active-low
<code>LED0_R</code>	O	Signal pilote (<i>driver</i>) LED0 canal rouge
<code>LED0_G</code>	O	Signal pilote (<i>driver</i>) LED0 canal vert
<code>LED0_B</code>	O	Signal pilote (<i>driver</i>) LED0 canal bleu
<code>LED1_R</code>	O	Signal pilote (<i>driver</i>) LED1 canal rouge
<code>LED1_G</code>	O	Signal pilote (<i>driver</i>) LED1 canal vert
<code>LED1_B</code>	O	Signal pilote (<i>driver</i>) LED1 canal bleu

Circuit

Le circuit modifié est décrit Fig. 5.

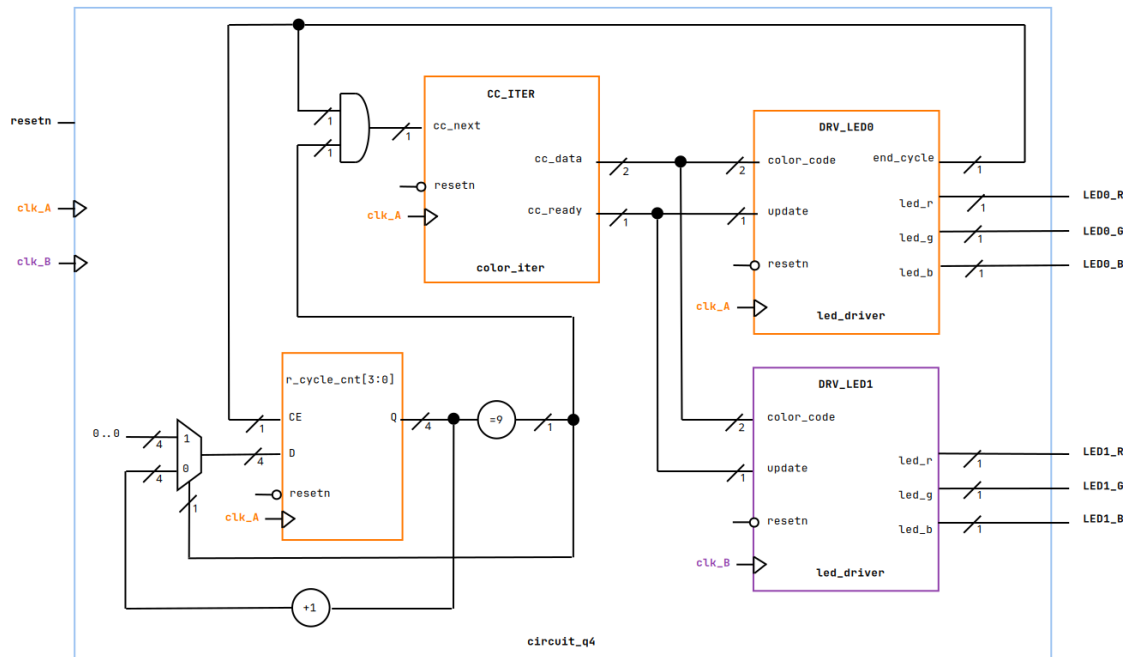


Figure 5: Circuit Question 4.

Architecture VHDL

Sources VHDL:

- Description du circuit Question 4: `src/design/circuit_q4.vhd`
- Codes couleurs (*package*): `src/design/color_codes_pkg.vhd`

L'architecture de l'entité `circuit_q4` décrit littéralement le circuit Fig. 5.

Simulation

Testbench:

- UUT: `circuit_q4`, $f_{clkA} = f_{clkB}$
- VHDL: `src/sim/tb_circuit_q4.vhd`
- Configuration *waveform*: `src/sim/tb_circuit_q4.wcfg`
- Paramètre $D = 10$, $K = 5$.

On vérifie rapidement sur le chronographe Fig. 6 que le comportement du circuit ne change pas lorsque les horloges `clk_A` et `clk_B` ont même fréquence.

Le changement de couleur des deux LEDs a lieu lorsque la LED0 a clignoté 10 fois.

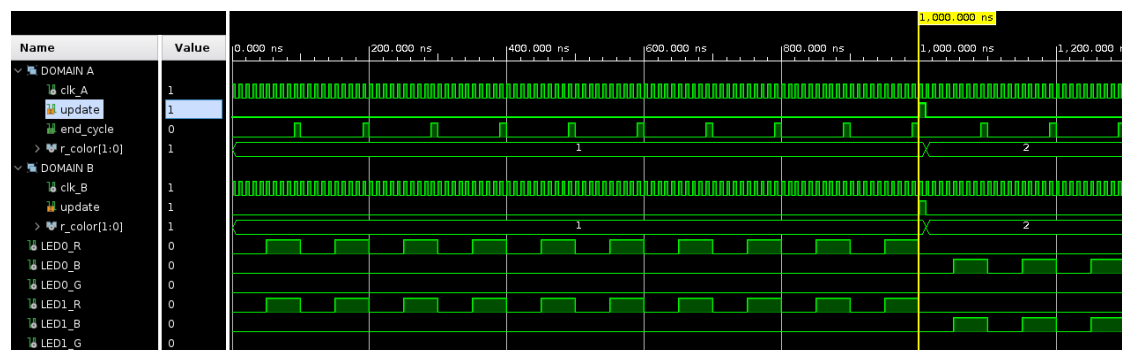


Figure 6: Chronographe `tb_circuit_q4`, $f_{clkA} = f_{clkB}$.

Question 5

Modifiez votre testbench tel que l'horloge `clkA` ait une fréquence de 250Mz et `clkB` 50MHz.

Testbench

Testbench:

- UUT: `circuit_q4`, $f_{clkA} = 5 \times f_{clkB}$
- VHDL: `src/sim/tb_circuit_q5.vhd`
- Configuration *waveform*: `src/sim/tb_circuit_q5.wcfg`
- Paramètre $D = 10$, $K = 5$.

Question 6

Lancer une simulation. Que se passe-t-il au niveau des signaux `update` des modules `LED_driver` ? Qu'elle incidence cela a-t-il sur les LEDs ?

Simulation

La période de l'horloge `clk_A` du domaine *A* est $\frac{1}{250.10^6} = 4$ ns. La période de l'horloge `clk_B` du domaine *B* est 20 ns.

Ainsi, du port de sortie `cc_ready` de `CC_ITER` dans le domaine *A*, au port d'entrée `update` de `DRV_LED1` dans le domaine *B*, la fréquence d'échantillonnage⁵ est divisée par 5. Il est difficile d'espérer *observer* un signal non ambiguë (Nyquist).

Il s'agit d'une situation de *Fast-to-Slow Clock Domain Crossing*: un signal produit dans le domaine avec la fréquence la plus élevée peut ne pas être *observable* dans la logique séquentielle d'un domaine de moindre fréquence.

Effectivement, sur le chronographe Fig. 7:

- Le signal `update` est *depuis longtemps* retourné au niveau bas (*de-asserted*) lors du front d'horloge montant suivant dans le domaine *B*: l'impulsion n'y est pas *vue*.
- Ainsi, dans le domaine *B* la LED rouge continuera de clignoter, tandis que la couleur passe au bleu dans le domaine *A*.

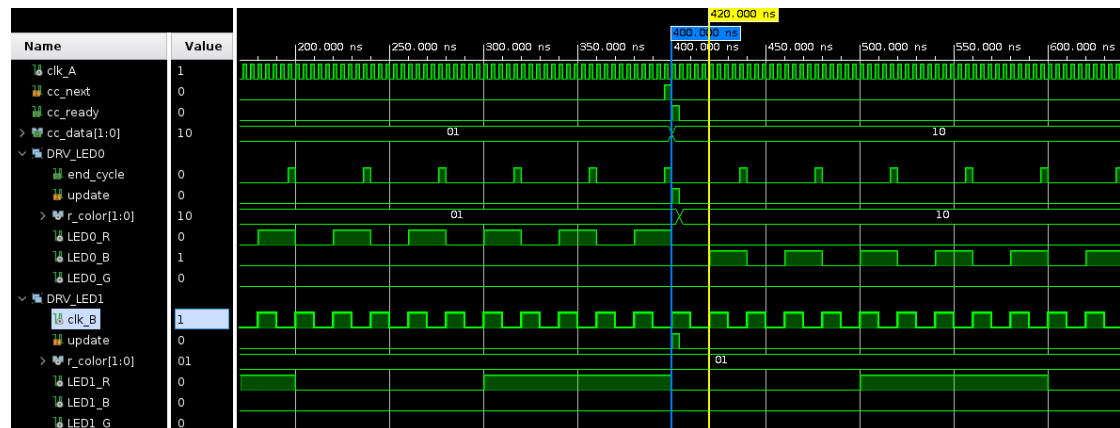


Figure 7: *Fast-to-Slow Clock Domain Crossing*, $f_{clkA} = 5 \times f_{clkB}$.

⁵La fréquence des transferts RTL $Q \leq D$.

Question 7

Proposez une solution pour corriger le problème lié au changement de domaine d'horloge, vous pourrez vous aider du lien <https://nandland.com/lesson-14-crossing-clock-domains/>.

Pulse Stretcher

Pour transférer l'impulsion `cc_ready` du domaine *A* vers le port d'entre `update` dans le domaine *B*, une approche est d'étirer (*stretch*) l'impulsion du domaine de plus haute fréquence pour qu'elle puisse être échantillonnée dans le domaine basse fréquence. C'est à dire un étirement $T_{stretch} = 2 \times T_{clkB}$.

Et, pour une horloge `clk_A` cinq fois plus rapide que `clk_B`, $T_{stretch} = 10 \times T_{clkA}$ comme illustré Fig. 8.

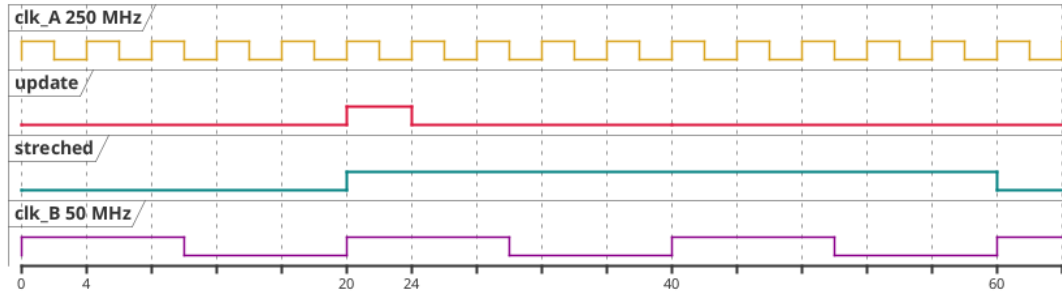


Figure 8: *Pulse Stretcher*.

Question 8

Mettez en place votre solution et testez-la en simulation. Si votre résultat de simulation n'est toujours pas valide, proposez une autre solution.

Architecture RTL

L'insertion d'un tel *pulse stretcher* $T_{stretch} = 10 \times T_{clkA}$ dans notre architecture est décrite Fig. 9.

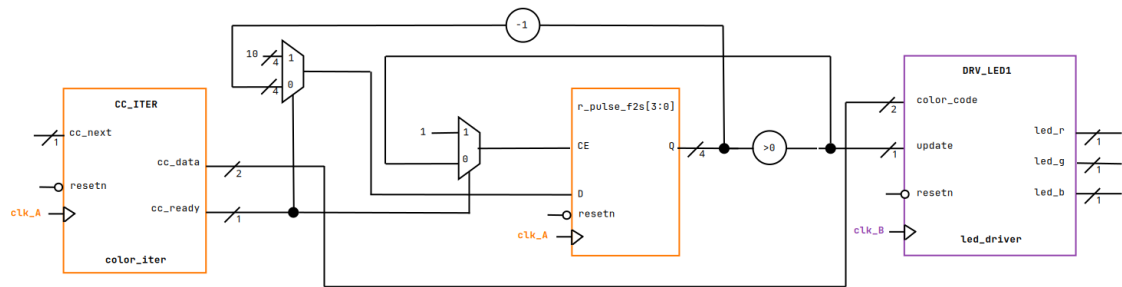


Figure 9: Circuit Question 7 (*Pulse Strecher*).

Le circuit introduit une nouveau paramètre configurant la longueur de l'impulsion étirée en nombre de périodes T_{clkA} .

Table 5: Paramètre *Pulse Stretcher*.

Générique	Configuration
F2F_PULSE_LEN	Étirement $\frac{T_{stretch}}{T_{clkA}}$

Architecture VHDL

Sources VHDL:

- Description du circuit Question 7: `src/design/circuit_q7.vhd`
- Codes couleurs (*package*): `src/design/color_codes_pkg.vhd`

Simulation

Testbench:

- UUT: `circuit_q7`, $f_{clkA} = 5 \times f_{clkB}$
- VHDL: `src/sim/tb_circuit_q7.vhd`
- Configuration *waveform*: `src/sim/tb_circuit_q7.wcfg`
- Paramètre $D = 10$, $K = 5$.

On vérifie:

- L'impulsion `update` en entrée de `DRV_LED0` dure toujours une période d'horloge `clk_A`.
- L'impulsion `update` en entrée de `DRV_LED1` est étirée pour durer deux périodes d'horloge `clk_B`, Fig. 10.
- Les changements de codes couleur sont bien visibles dans les deux domaines *A* et *B*.
- Les changements de couleur des deux LEDs ont lieu lorsque la `LED0` à clignoté 10 fois, Fig. 11.

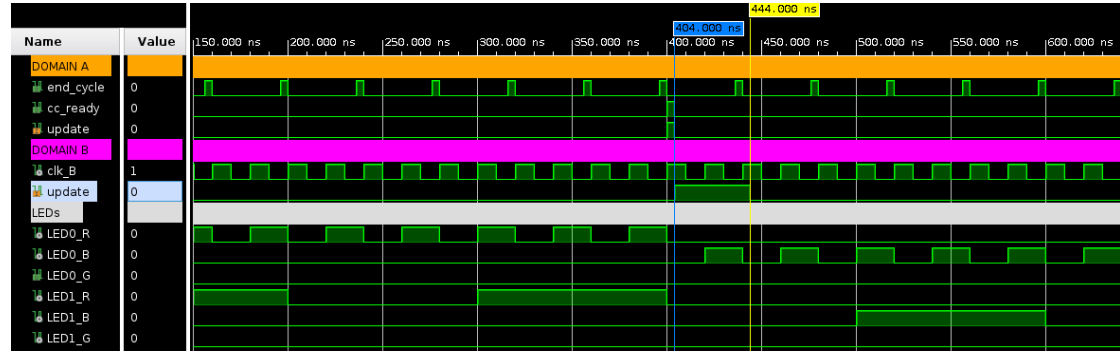


Figure 10: Impulsion `update` étirée, $T_{stretch} = 2 \times f_{clkB}$.

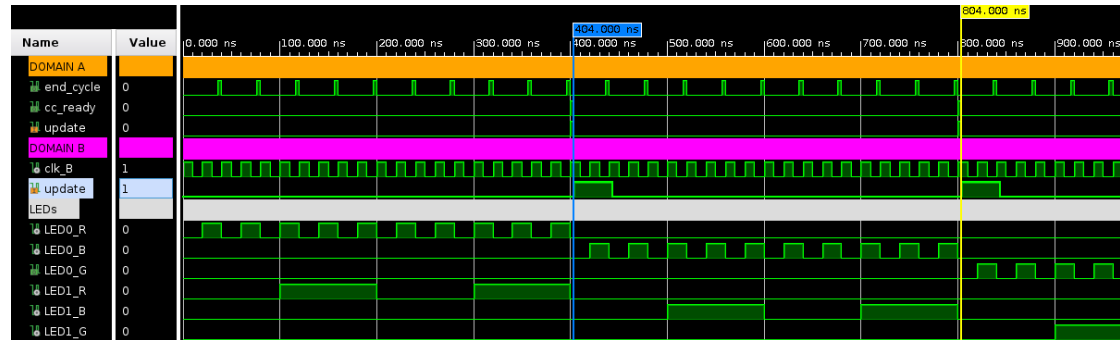


Figure 11: Chronographe `tb_circuit_q7`, $f_{clkA} = 5 \times f_{clkB}$.

Question 9

Pour générer plusieurs horloges vous aurez besoin d'une PLL. Trouvez quel est le nom de l'IP PLL de l'IP Catalog de Vivado puis ajoutez là à votre architecture. La fréquence de l'horloge en entrée de la PLL est de 100MHz. Les deux horloges en sortie doivent être à 50MHz et 250MHz.

IP PLL

L'horloge source est l'horloge du sous-système PL, 100 MHz.

Le catalogue Vivado propose l'IP Clock Wizard 6, qui peut synthétiser des horloges de différentes fréquences à partir d'une source, avec alignement des phases.

Table 6: Configuration PLL Clock Wizard.

Clock	Type	Fréquence
clk	Input (PL)	100 MHz
clk_A	PLL Output	250 MHz
clk_B	PLL Output	50 MHz

Architecture RTL

Les ports I/O de l'IP Clock Wizard sont donnés Table 7.

Table 7: Ports I/O Clock Wizard.

Port	Type	Signal
clk	I	Horloge PL
clk_A	O	Horloge domaine A
clk_B	O	Horloge domaine B

Architecture VHDL

Sources VHDL:

- Codes couleurs (*package*): `src/design/color_codes_pkg.vhd`
- Description du circuit Question 9: `src/design/circuit_q9.vhd`

Question 10

Effectuez la synthèse et placez des sondes (ILA) sur les signaux pertinents pour vérifier que le changement de domaine d'horloge s'est correctement passé.

Sondes ILA

Le franchissement de domaine qui nous intéresse est celui du signal `update`, pilotant les changements de couleurs:

- Directement connecté au stimulus `cc_ready` dans le domaine *A*.
- Étiré par le *Pulse Stretcher* dans le domaine *B*.

Ces changements de couleur doivent apparaître:

- Tous les dix clignotements de la LED du domaine *A*.
- Tous les deux clignotements de la LED du domaine *B*, dont l'horloge est cinq fois moins rapide.

C'est ce dernier point que nous vérifierons en plaçant des sondes *debug* sur les sorties LED du domaine *B*:

- Sondes `LED1_R_OBUF`, `LED1_B_OBUF`, `LED1_G_OBUF`.
- Domaine d'horloge `clk_B`.
- *Trigger* pour fronts montants et descendants (`==B`)
- Window data depth: 2
- Trigger position in window: 0
- Number of windows: quelques unes, disons 32

Question 11

Etudiez le rapport de synthèse.

Rapports d'utilisation

- Utilization Design Information: `synth/circuit_q9_utilization_synth.rpt`
- Synthesis report: `synth/circuit_q9.vds`

Table 8: Summary of Registers by Type.

Total	Clock Enable	Synchronous	Asynchronous
3	Yes	-	Set
68	Yes	-	Reset

Table 9: Primitives (synthèse).

Ref Name	Used	Functional Category
FDCE	68	Flop & Latch
LUT5	62	LUT
LUT6	16	LUT
CARRY4	14	CarryLogic
LUT3	10	LUT
LUT4	9	LUT
OBUF	6	IO
FDPE	3	Flop & Latch
LUT1	2	LUT
LUT2	1	LUT
IBUF	1	IO

Nous avons, lors du TP4 - Partie 1, observé qu'une unité `led_driver` synthétise une trentaine de registres: 27 FDCE pour le compteur de type `counter_unit`, plus deux FDCE et un FDPE pour l'unité `led_driver` elle-même.

Soit 58 FDCE et 2 FDPE pour les deux instances `led_driver`.

Le circuit synthétisé ici introduit:

- Une unité `color_iter`: 2 FDCE (MSB `cc_data` et `cc_ready`), et 1 FDPE (LSB `cc_data`).
- Une unité `circuit_q9`: 4 FDCE `r_cycle_cnt`, plus 4 FDCE `r_pulse_f2s`.

Soit effectivement 68 FDCE et 3 FDPE.

Un seul IBUF (`c1k`), et 6 OBUF (les canaux RGB des deux LEDs).

Les ressources utilisées par l'IP PLL Vivado sont données Table 10.

Table 10: Primitives IP PLL (synthèse).

Ref Name	Used	Functional Category
BUFG	3	Clock
PLLE2 _{ADV}	1	Clock
IBUF	1	IO

Rapport de *Timing* (STA)

- Timing Summary Report: `synth/timing_report.txt`
- Design Analysis Report: `synth/design_analysis_report.txt`

L'analyse statique Table 11 produit des résultats TNS (Setup) et WNS (Hold) négatifs.

Table 11: Design Timing Summary.

WNS(ns)	TNS(ns)	WHS(ns)	THS(ns)	WPWS(ns)	TPWS(ns)
-0.538	-4.610	-0.013	-0.026	1.5	0.000

Les rapports montrent bien deux types de violations:

- *Setup* pour des chemins intra-domaine *A*: la limitation est ici le *design* VHDL du compteur de temporisation `counter_unit`, qu'il nous faudra corriger pour passer les tests de *timing* avec une fréquence d'horloge de 250 MHz.
- *Hold* pour des chemins inter-domaines: l'analyse semble ne pas avoir pleinement reconnu le *Pulse Stretcher*.

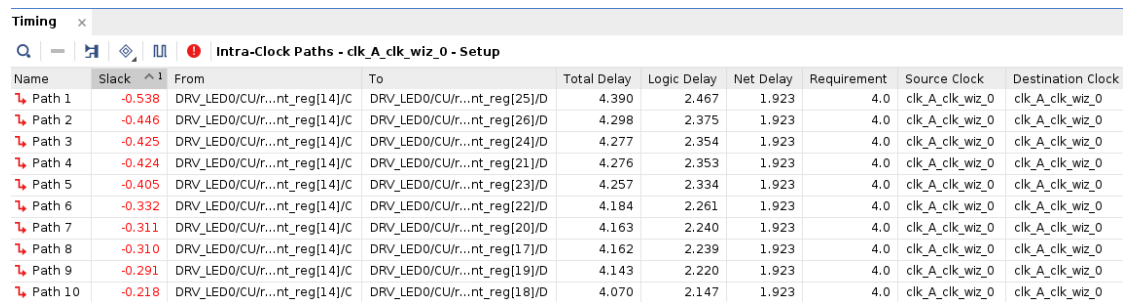
Intra-domaine, violations Setup

Les chemins causant ces violations aboutissent au compteur de temporisation de l'unité `DRV_LED0`, à l'intérieur du domaine d'horloge `clk_A`, 250 MHz.

Synthétiser l'incrémentation d'un compteur 27-bit, ou sa comparaison avec une valeur de même magnitude, produits des chemins RTL qui peuvent se révéler coûteux (*longs*) et ne plus être *négligeables* à haute fréquence (ici 250 MHz).

Le *design* VHDL du *compteur de temporisation* place de plus la comparaison avec le modulo sur le chemin de la logique de sortie `end_counter`, qui, *in fine*, est le signal pulsation de l'ensemble du circuit.

Modifier ce *design* VHDL, en pré-calculant et *cachant* la comparaison en dehors de la logique de sortie `end_counter`, suffit à éliminer les violations *Setup* rapportées. Le *patch* appliqué ici est donné en Annexe.



Name	Slack	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 1	-0.538	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[25]/D	4.390	2.467	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 2	-0.446	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[26]/D	4.298	2.375	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 3	-0.425	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[24]/D	4.277	2.354	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 4	-0.424	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[21]/D	4.276	2.353	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 5	-0.405	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[23]/D	4.257	2.334	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 6	-0.332	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[22]/D	4.184	2.261	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 7	-0.311	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[20]/D	4.163	2.240	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 8	-0.310	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[17]/D	4.162	2.239	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 9	-0.291	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[19]/D	4.143	2.220	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0
Path 10	-0.218	DRV_LED0/CU/r...nt_reg[14]/C	DRV_LED0/CU/r...nt_reg[18]/D	4.070	2.147	1.923	4.0	clk_A_clk_wiz_0	clk_A_clk_wiz_0

Figure 12: Violations intra-domaine, *Setup*.

Inter-domaines, violations Hold

Ces violations sont associées à des chemins RTL *Fast-to-Slow*, du domaine d'horloge `clk_A` vers le domaine `clk_B`, et pourraient révéler un état *métastable* d'un registre du domaine *A* au moment de l'enregistrement de sa sortie dans le registre du domaine *B* `r_color`.

Or nous savons que le registre `cc_data` du domaine *A* en entrée de `r_color` sera stable pendant 10×2.10^8 périodes d'horloge `clk_A`, soit 4.10^8 périodes d'horloge `clk_B` après assertion du signal `update`. L'enregistrement d'un niveau métastable semble donc largement improbable.

L'analyse statique semble par ailleurs ne pas avoir pleinement reconnu le *Pulse Stretcher*, et le rôle du registre `r_pulse_f2s`.

Ces violations s'évanouissent lors du placement & routage, et ne retiendront pas davantage notre attention.

Timing x									
Inter-Clock Paths - clk_A_clk_wiz_0 to clk_B_clk_wiz_0 - Hold									
Name	Slack	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destination Clock
Path 43	-0.013	r_pulse_f2s_reg[3]/C	DRV_LED1/r_color_reg[0]/D	0.447	0.239	0.208	0.0	clk_A_clk_wiz_0	clk_B_clk_wiz_0
Path 44	-0.013	r_pulse_f2s_reg[3]/C	DRV_LED1/r_color_reg[1]/D	0.447	0.239	0.208	0.0	clk_A_clk_wiz_0	clk_B_clk_wiz_0

Figure 13: Violations inter-domaines, *Hold*.

Question 12

Effectuez le placement routage et étudiez les rapports générés.

Implémentation

Fichiers de contraintes: `src/const/cdc.xdc`

Placement & Routage

- Utilization - Place Design: `impl/circuit_q9_utilization_placed.rpt`

Les ressources utilisées après placement & routage correspondent à celles synthétisées.

Les deux registres supplémentaires (70 FDCE et non 68) que l'on pourrait remarquer sont ajoutés par le *patch* appliqué précédemment pour corriger le *design* VHDL du module `counter_unit`.

Table 12: Summary of Registers by Type.

Total	Clock Enable	Synchronous	Asynchronous
3	Yes	-	Set
70	Yes	-	Reset

Rapports de Timing (STA)

- Timing Summary Report: `impl/timing_report.txt`

L'implémentation ne révèle pas de violation *Setup Time* ni *Hold Time*, ce qui semble valider l'approche adoptée lors de l'analyse des rapports de synthèse.

Table 13: Design Timing Summary.

WNS(ns)	TNS(ns)	WHS(ns)	THS(ns)	WPWS(ns)	TPWS(ns)
0.767	0.000	0.170	0.000	1.5	0.000

Question 13

Générez le bitstream, programmez la carte et vérifiez les signaux du chipscope (ILA).

Bitstream

- Bitstream (2 MB): `impl/circuit_q9.bit`

ILA

La forme des ondes instrumentées pour les LEDs de sortie du domaine *B* est donnée Fig. 14.

On peut vérifier que la périodicité des changements de couleur, c'est à dire 10 clignotements dans le domaine *A*, n'est plus que de deux clignotements dans le domaine *B* d'horloge cinq fois moins rapide.

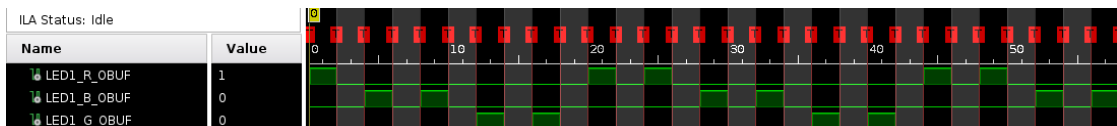


Figure 14: Sondes ILA LED1

Démonstration

Scénario de démonstration vérifié sur carte Cora Z7 (`xc7z010c1g400-1`):

- Au démarrage, les deux LEDs rouges clignotent.
- Lorsque la LED rouge du domaine A a clignoté 10 fois, et la LED rouge du domaine B deux fois, les deux LEDs bleues clignotent.
- Lorsque la LED bleue du domaine A a clignoté 10 fois, et la LED bleue du domaine B deux fois, les deux LEDs vertes clignotent.
- Lorsque la LED verte du domaine A a clignoté 10 fois, et la LED verte du domaine B deux fois, les deux LEDs rouges clignotent.

Patch counter_unit

L'approche ⁶ est de pré-calculer et enregistrer la coûteuse comparaison $r_count + 1 = K - 1$ pour la retirer de logique de sortie `end_counter`.

```
--- counter_unit.vhd      2026-09-03 21:06:48.689877525 +0200
+++ counter_unit_sta_fix.vhd  2026-09-03 21:00:34.343536814 +0200
@@ -47,28 +47,36 @@
-- FIXME: initial state should rely on Power-on Reset signal.
signal r_count  : unsigned(N-1 downto 0) := (others => '0');

- -- Wired to the combinatorial logic for r_count = K - 1.
- signal w_modK  : std_logic;
+ -- Pre-compute and register condition (r_count + 1 = K - 1)
+ -- to avoid some delay on end_counter output logic.
+ signal r_modK  : std_logic := '0';

begin

- -- Combinatorial circuit.
- -- Evaluated on rising edges, when r_count is updated.
- w_modK <= '1' when (r_count = K - 1) else '0';
- end_counter <= w_modK;
+ -- Pre-computed, no output logic.
+ end_counter <= r_modK;

rtl: process(clk, resetn)
begin

    if (resetn = '0') then
        r_count <= (others => '0');
+       r_modK <= '0';

    elsif rising_edge(clk) then
-       if (w_modK = '1') then
+       -- r_count + 1 = K - 1, avoiding overflows.
+       if (r_count = K - 2) then
+           r_modK <= '1';
+       else
+           r_modK <= '0';
+       end if;
+
+       if (r_modK = '1') then
            r_count <= (others => '0');
        else
            r_count <= r_count + 1;
        end if;
+
    end if;

end process rtl;
```

⁶Pour appliquer le *patch*: `cd src/design && patch counter_unit.vhd < counter_unit_sta_fix.patch`